

Dry run Sieve algorithm

Let's dry run the **Sieve of Eratosthenes** algorithm step by step.

Suppose:

```
C++
n = 20
```

Step 1: Create the array

```
C++
vector<bool> isPrime(21, true);
```

Initially every number is assumed to be prime.

Number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Prime?	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T

Step 2: Mark 0 and 1

```
C++
isPrime[0] = false;
isPrime[1] = false;
```

Number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Prime?	F	F	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T

Step 3: Start outer loop

```
C++
for(int i = 2; i*i <= 20; i++)
```

Notice

```
sqrt(20) ≈ 4.47
```

So i will be

```
2
3
4
```

Iteration 1

```
i = 2
```

Check

```
C++
if(isPrime[2])
```

Yes, it is prime.

Now run

```
C++
for(int j = 2*2; j <= 20; j += 2)
```

Starts from

```
j = 4
```

Marks

```
4
6
8
```

```
10
12
14
16
18
20
```

as false.

Array becomes

Number	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Prime?	T	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F

Iteration 2

```
i = 3
```

Check

```
C++
isPrime[3]
```

Still true.

Now mark multiples.

Starts from

```
C++
j = 3*3 = 9
```

Why not from 6?

We'll explain later.

Now mark

```
9
12
15
18
```

Array becomes

Number	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Prime?	T	T	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	F

Notice

```
12
18
```

were already false.

No problem.

Iteration 3

```
i = 4
```

Check

```
C++
isPrime[4]
```

It is already false.

That means

```
4 is not prime.
```

So we skip

```
C++
```

```
for(...)
```

because all multiples of 4 were already removed while processing 2.

Outer loop ends

Next

```
i = 5
```

Condition

```
5*5 <= 20
```

is false.

Loop stops.

Remaining true values

```
2
3
5
7
11
13
17
19
```

These are exactly the prime numbers ≤ 20 .

Why start from $i*i$?

Take

```
i = 5
```

Multiples are

```
5*2 =10
5*3 =15
5*4 =20
5*5 =25
5*6 =30
...
```

Look carefully.

```
10
```

was already removed by 2.

```
15
```

was already removed by 3.

```
20
```

was already removed by 2.

So the first number that hasn't necessarily been processed is

```
5*5 =25
```

Hence

```
C++
j = i*i;
```

avoids doing unnecessary work.

Why only go until $\text{sqrt}(n)$?

Suppose

```
n = 100
```

Do we need to process

```
11
12
13
...
100?
```

No.

Because every composite number has a factor less than or equal to $\sqrt{n}$.

Example

```
91 = 7 × 13
```

Factor 7 is already $\leq \sqrt{100}$.

Similarly

```
95 = 5 × 19
```

Factor 5 already removed it.

Once every prime $\leq \sqrt{n}$ has marked its multiples, every composite number has been crossed out.

Complete visualization

Initially

```
2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
T T T T T T T T T T T T T T T T T
```

i=2

Remove:

```
4 6 8 10 12 14 16 18 20
```

```
2 3 5 7 9 11 13 15 17 19
```

i=3

Remove:

```
9 12 15 18
```

```
2 3 5 7 11 13 17 19
```

i=4

Skip (already composite)

Done.

Time Complexity

- **Time:** $O(n \log \log n)$
- **Space:** $O(n)$

The reason this is so efficient is that each composite number is marked only by its prime factors, and starting from $i*i$ avoids redundant markings.

Let's derive the **time complexity of the Sieve of Eratosthenes** mathematically, step by step.

Code

```
C++
for (int i = 2; i * i <= n; i++) {
    if (isPrime[i]) {
        for (int j = i * i; j <= n; j += i) {
            isPrime[j] = false;
        }
    }
}
```

There are **two loops**, so we'll analyze each one.

Step 1: Outer Loop

```
C++
for (int i = 2; i * i <= n; i++)
```

Condition is

```
C++
i * i <= n
```

Taking square root,

```
i <= √n
```

Therefore the outer loop runs

```
√n times
```

So,

$$T_{outer} = O(\sqrt{n})$$

But this **is not** the final complexity because the inner loop does different amounts of work for each **i**.

Step 2: Inner Loop

Suppose

```
C++
i = 2
```

Then

```
C++
for(j = 4; j <= n; j += 2)
```

The values are

```
4
6
8
10
...
n
```

Number of iterations is approximately

$$\frac{n}{2}$$

Now suppose

```
C++
i = 3
```

Loop is

```
C++
j = 9, 12, 15, ...
```

Iterations

$$\frac{n}{3}$$

For

```
C++
i = 5
```

Iterations

$$\frac{n}{5}$$

For

```
C++
```

$i = 7$

Iterations

$$\frac{n}{7}$$

Generally,

for prime p ,

iterations are approximately

$$\frac{n}{p}$$

Step 3: Add the Work

Ignoring constants,

Total work becomes

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \dots$$

Factor out n ,

$$n \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots \right)$$

Now the question becomes

What is

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots$$

where the sum is over **primes**?

Step 4: Prime Harmonic Series

A famous result in number theory says

$$\sum_{p \leq n} \frac{1}{p} = \log \log n + O(1)$$

This means

$$\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \dots = \log \log n$$

approximately.

Step 5: Multiply Back

Therefore

$$T(n) = n \cdot \log \log n$$

Hence

$$O(n \log \log n)$$

Why not $O(n\sqrt{n})$?

A common mistake is saying:

- Outer loop runs $\sqrt{n}$ times.
- Inner loop runs n/i times.
- Therefore,

$$\sqrt{n} \times n = n\sqrt{n}$$

This is **incorrect** because the inner loop **does not always run n times**. It runs:

i	Iterations
2	$n/2$
3	$n/3$
5	$n/5$
7	$n/7$
11	$n/11$
13	$n/13$

The number of iterations keeps decreasing as **i** increases, so you **sum** the costs instead of multiplying them.

Example for $n = 100$

The primes up to $\sqrt{100} = 10$ are:

2, 3, 5, 7

Work done:

Prime	Iterations
2	$100/2 = 50$
3	$100/3 \approx 33$
5	$100/5 = 20$
7	$100/7 \approx 14$

Total

$50 + 33 + 20 + 14 = 117$

This matches

$$100 \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} \right) \approx 117$$

Notice it's **much smaller** than

$100 \times \sqrt{100} = 1000$

which is why the complexity is **not** $O(n\sqrt{n})$.

Intuition

The sieve is fast because:

1. **Only prime numbers** perform marking. Composite numbers are skipped due to `if (isPrime[i])`.
2. **Each prime marks only its multiples**, which is about n/p operations.
3. The sum of the reciprocals of the primes grows extremely slowly, like $\log \log n$, leading to the final complexity:

$$\text{Time Complexity} = O(n \log \log n)$$

and the space complexity is:

$$O(n)$$

This is a very good question. The confusion is:

If the outer loop runs only $\sqrt{n}$ times, why does the final complexity become $O(n \log \log n)$ instead of $O(\sqrt{n} \log \log n)$?

The answer is **because the dominant work is done by the inner loop**, not the outer loop.

Let's understand it carefully.

Step 1: Outer loop

C++

```
for(int i = 2; i * i <= n; i++)
```

The number of iterations is

$$\sqrt{n}$$

For example, if

```
n = 100
```

then

```
i = 2,3,4,5,6,7,8,9,10
```

Only **9 iterations**.

So the outer loop itself costs

$$O(\sqrt{n})$$

Step 2: Inner loop

Suppose

```
n = 100
```

When

```
i = 2
```

the inner loop runs

```
4
6
8
...
100
```

≈ **50 iterations**

When

```
i = 3
```

it runs

```
9
12
15
...
99
```

≈ **33 iterations**

When

```
i = 5
```

≈ **20 iterations**

When

```
i = 7
```

≈ **14 iterations**

Total work is

```
50 + 33 + 20 + 14
```

Notice something important:

The **inner loop alone has already done 117 operations**, while the outer loop only executed **9 times**.

The outer loop is almost negligible compared to the work done inside it.

Step 3: Write the sum

The total work is

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \dots$$

Factor out n :

$$n \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots \right)$$

The remaining sum is

$$\log \log n$$

So

$$T(n) = n \log \log n$$

Why don't we multiply by $\sqrt{n}$?

Many beginners think:

```
Outer loop =  $\sqrt{n}$ 
Inner loop =  $n/i$ 
Answer =  $\sqrt{n} \times n$ 
```

This is **incorrect**.

Why?

Because the inner loop **doesn't run n/i for every number from 2 to $\sqrt{n}$** in the same amount. Each iteration has a **different cost**.

Think of it like this:

i	Inner loop work
2	50
3	33
5	20
7	14

You **add** these costs:

```
50 + 33 + 20 + 14
```

You do **not** multiply:

```
4 × 50
```

because each outer-loop iteration performs a different amount of work.

General Rule for Nested Loops

If the inner loop always runs the same number of times:

```
C++
for(i=1; i<=n; i++)
    for(j=1; j<=n; j++)
```

then

$$O(n) \times O(n) = O(n^2)$$

because every outer iteration does exactly n inner iterations.

But if the inner loop depends on i :

```
C++
for (i=1; i<=n; i++)
    for (j=i; j<=n; j++)
```

the work is

$$(n) + (n-1) + (n-2) + \dots + 1$$

which is a **sum**, not a multiplication.

Similarly, in the sieve:

```
C++
for (i=2; i*i<=n; i++)
```

the work is

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \dots$$

which simplifies to

$$O(n \log \log n)$$

Key takeaway

- The **outer loop** only determines **how many different inner loops** are executed ($\sqrt{n}$ of them).
- The **total running time** is the **sum of the work done by all those inner loops**, not $\sqrt{n}$ multiplied by the maximum inner-loop cost.
- Since the sum of the inner-loop work is $n \log \log n$, that term dominates the $\sqrt{n}$ cost of the outer loop.

So the final complexity is:

$$O(n \log \log n)$$

not

$$O(\sqrt{n} \cdot n).$$

ChatGPT can make mistakes. Check important info.